Generative AI Usage and Development Notes

Course: EECS 581 – Software Engineering II

Assignment: A1 – Extracting IPv4 Addresses from Noisy Text

Student: Fabrizio Falcon

AI Tool: ChatGPT – GPT-5.6 Sol, OpenAI

September 26, 2026 – Initial Use

I used ChatGPT after reading the assignment to make sure I understood the parsing requirements and the restrictions placed on the implementation.

Instead of treating the assignment as a simple code-generation task, I used the model first to break down the specification and identify cases that could cause problems.

Prompting Approach

I used ChatGPT iteratively, first to clarify the specification and then to produce and review an implementation under the assignment constraints.

Prompt Used:

> Analyze this assignment specification before writing code. Identify the parsing rules, prohibited library functions, edge cases, and any ambiguity around what counts as one complete candidate token. I want the implementation to follow the specification literally rather than trying to recover partial matches.

After establishing the requirements, I used a more implementation-focused prompt:

> Write a C++ solution using the required extractIPv4 prototype. Parse everything character by character and manually accumulate numeric values. Do not use regex, stoi, atoi, strtol, sscanf, or networking parsing functions. Treat each uninterrupted run of digits, periods, and colons as a complete candidate, and reject the whole run if any part of the IPv4 address or optional port is malformed.

I then used the model as a reviewer instead of assuming its generated code was correct:

> Audit this implementation specifically against adversarial inputs. Look for cases where it might accidentally skip punctuation, accept a valid prefix from a malformed longer token, ignore an invalid port, allow leading zeros, or mishandle boundary values such as 255 and 65535. Do not just tell me whether it compiles; compare the behavior directly against the assignment rules.

September 27, 2026 – Testing and Review

While reviewing the generated code, I found an issue in the way the first version searched for candidate addresses.

The original version skipped characters until it found a digit and then started parsing from that position.

At first this looked reasonable, but it did not fully follow the assignment specification.

For example:

```
.1.2.3.4
```

The first AI-generated implementation could skip the initial period and then accept:

```
1.2.3.4
```

This was incorrect because the assignment states that a stray period or colon directly adjacent to an otherwise valid address must cause the complete candidate to be rejected.

The code compiled and looked fine at first glance, but the logic was still wrong. This was a good example of why I did not want to trust the LLM just because it produced clean-looking code.

Correction

I corrected the scanner so that it does not simply search for the next digit.

A candidate now begins whenever the scanner encounters any character that is allowed to belong to a token:

```
0-9
.
:
```

The scanner then collects the complete uninterrupted sequence of those characters and validates the entire candidate.

Because of this change:

```
.1.2.3.4
```

is rejected as a whole.

Likewise:

```
1.2.3.4.
```

is rejected instead of extracting only the valid-looking prefix.

The same rule also prevents:

```
1.2.3.4:99999
```

from being accepted as just:

```
1.2.3.4
```

because the invalid port makes the entire candidate invalid.

Additional Testing

I tested the final parser against both the examples from the assignment and additional boundary and malformed cases.

Examples included:

```
192.168.1.1
255.255.255.255
0.0.0.0
192.168.01.1
256.1.1.1
1.2.3.4:65535
1.2.3.4:65536
1.2.3.4:00
1.2.3.4:
1.2.3.4:80:90
1.2.3.4.5
1..2.3.4
```

I specifically checked:

- octets greater than 255
- ports greater than 65535
- leading zeros
- missing octets
- empty octets
- extra periods
- extra colons
- malformed ports
- addresses surrounded by unrelated text
- malformed candidates followed by a valid candidate later in the input

All numeric conversion is performed manually instead of using prohibited conversion functions.

The general accumulation logic is:

```
value = value * 10 + (digit - '0');
```

AI-Generated Portions

ChatGPT was used to generate the initial versions of:

- the extractIPv4 parsing logic
- the main input loop
- IPv4 decimal conversion
- output formatting
- test cases (personally checked each myself)
- README documentation
- AI-use documentation (read through everything and modified parts, used the original version mainly as template instead of the source of truth)

The final code attribution is for the LLM, it was not modified by me. I reviewed and tested the generated result instead of assuming that the first version was correct.

The candidate-scanning logic was specifically changed after testing exposed a case that the initial AI-generated implementation handled incorrectly.

Verification

I understand the submitted code and the purpose of each section of the parser.

I tested the final implementation using the examples provided in the assignment as well as additional boundary and malformed inputs.

At the time of submission, I do not know of any remaining bugs for the grammar specified by the assignment.

The main thing I learned from using the LLM on this assignment was that generated code can look convincing and still contain subtle logical errors, maybe hidden in edges cases that we don't think of at first. The first candidate-scanning implementation was a clear example of that.